Moca User & template lifecycle automation

Self-service workflows with policy-based gates and provenance tracking

2026-11-12

Outline

6 sections; the numbers are slide numbers; backup: slide 36

1 Why and what

motif, design, architecture

3 The brain thinks, the hands act; a template shapes the hands
4 Self-serve: policy, not a person, decides; every step is recorded
5 One control plane decides; hosts build and run; clients only ask

2 Users, and the demo

bootstrap; the demo's commands; API; roles

6 The settings name the first admins; the API manages everyone else
7 Demo: a user's lifecycle, in seven commands
8 Demo: a template's lifecycle, image to retire
9 Five routes, one mocactl command; each change is a record
10 A grant takes effect at the next request, and can expire by itself
11 Onboarding a user: one GitHub login, one enroll

3 Self-serve templates

create, refusals, profiles, records

12 Create to registered: no approver, every decision recorded
13 Refusals are answers with a rule, not silence
14 A create names a profile; the profile names the rules
15 Your own gate record counts for your own templates, privately
16 The client's whole path, in mocactl
17 Every step is a chained record, checkable offline

4 Using a template

a build's states, activation, sessions

18 A build has two states: how far from usable, and whether it runs now
19 Activation is a lease; the worker starts in seconds
20 First use: name a template and go
21 A Moca Pi session runs only on its build
22 One control plane serves both kinds of brain
23 The API covers the whole lifecycle, create to retire

5 Validation and speed

windows 1-4; cold and warm starts

24 Four windows validated the lifecycle end to end
25 Create to retire, through the API alone, on the cluster
26 Activation fell from ~100 s to ~11-17 s
27 Disk throughput sets the cold start: 3.1× faster
28 A warm worker activates in seconds: 7 s, not 980 s
29 28 cold starts at once queue behind the start limit
30 The fixed verify cache halves a cold start's reads

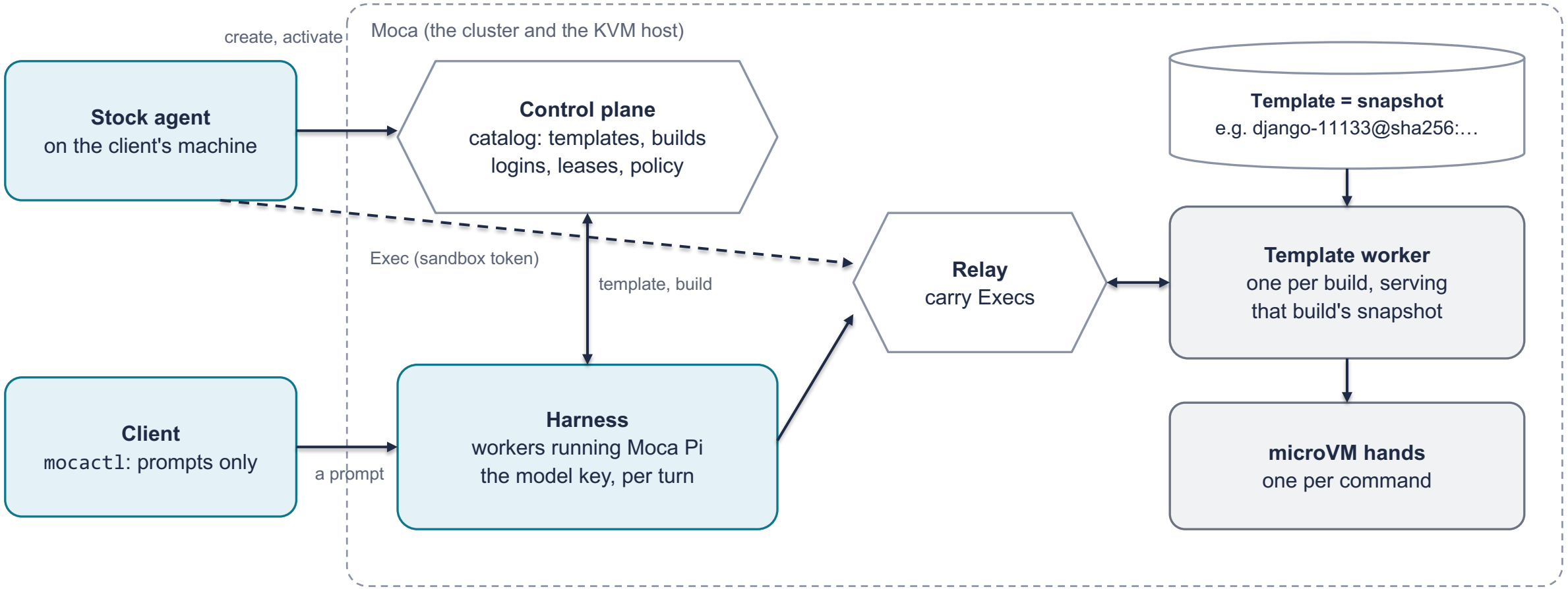
6 At scale, upstream, next

Lite-28, findings, PRs, next

31 SWE-bench Lite on templates, in groups, retire between
32 Two brains on one build, every link exact
33 What the windows found, and the fixes
34 The enhancements are open upstream, as reviewable PRs
35 Next: land the PRs upstream, then the demo

The brain thinks, the hands act; a template shapes the hands

Moca Pi brain, or a stock agent, with Moca microVM sandbox hands



The hands never hold the model key or the agent
they get commands and return output; a template decides what the hands contain

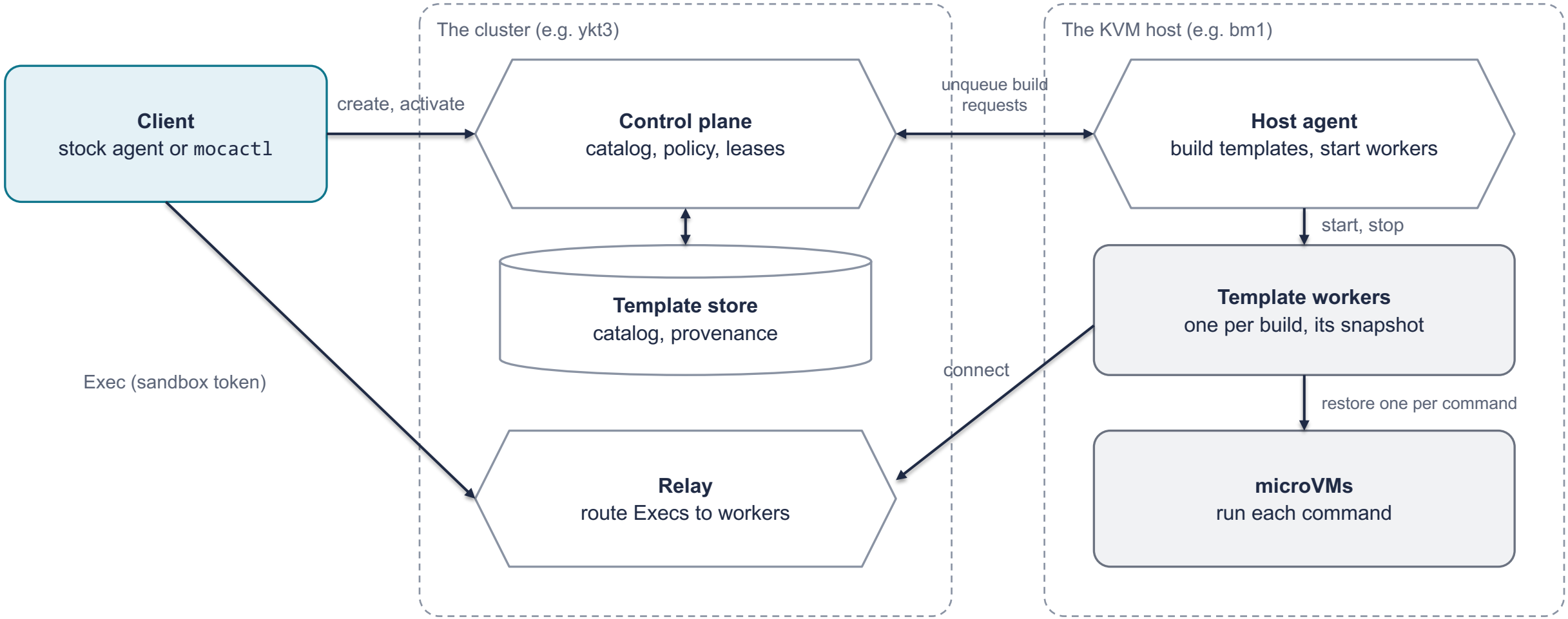
Self-serve: policy, not a person, decides; every step is recorded

The design in five ideas (template spec §3, §6-§8; enrollment spec §3)

Templates are declared in a store	a name over immutable builds, each named by its snapshot's manifest hash
Policy gates every image	rules checked at create; a build becomes `registered` by itself once its probe passes in a restored microVM and the rules hold again
Hosts build and run; the control plane decides	a host agent claims builds and starts workers; clients activate by lease
Users and roles through the API	the settings bootstrap the first admins; enroll, grant and revoke are API calls, evaluated per request
Every step is a chained record	append-only provenance named by digest, verifiable offline, off the cluster

One control plane decides; hosts build and run; clients only ask

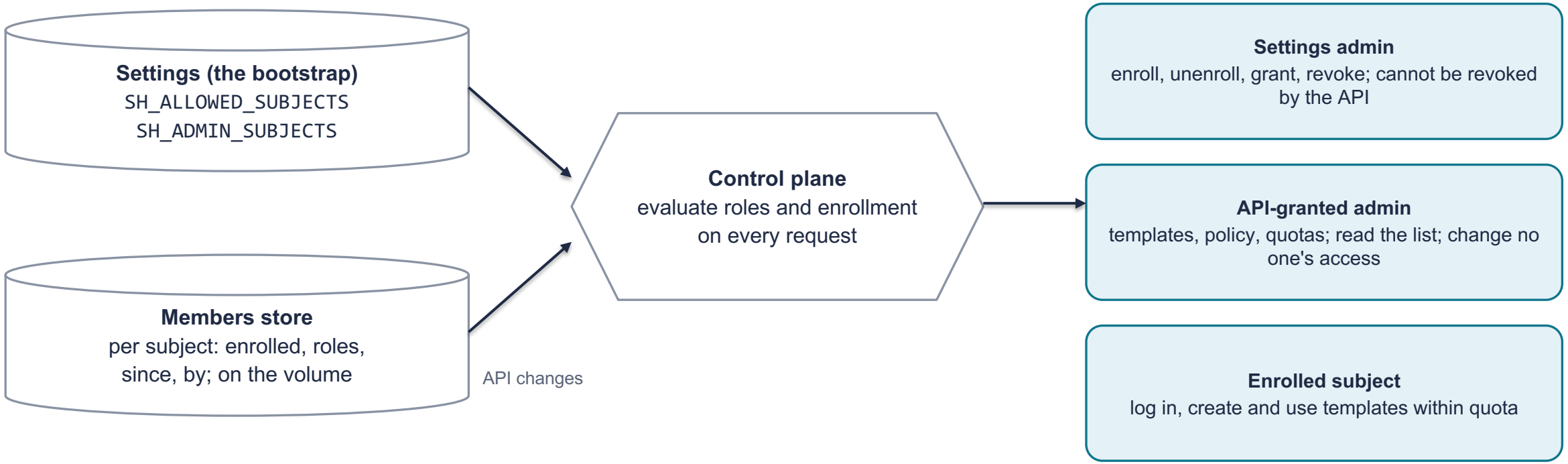
Who runs where: the cluster (OpenShift; e.g. ykt3) and the KVM host (bare metal; e.g. bm1)



A worker **connects** to the relay and holds one two-way gRPC stream (Attach): commands come down it, results go back up. Readiness is a worker connected to the relay; the cluster holds no VM, the KVM host no policy.

The settings name the first admins; the API manages everyone else

Who may use the service, and who is admin (enrollment spec §3, §6)



The recovery path stays the deployment's
the settings' admins are always admins: none can be revoked through the API, and none can revoke itself

Demo: a user's lifecycle, in seven commands

As the demo runs it: the admin on one laptop, the user on another (demo runbook §1, §5)

Who	Command	What the room sees
user	`mocactl sbx login`	GitHub's device code; `403 not_enrolled` until step 2
admin	`sbx admin subjects enroll github:<id>`	a `member` record; the user's next request works
admin	`sbx admin subjects`	each subject, its roles, and `env` or `api` as source
admin	`sbx admin subjects grant github:<id> --until 30m`	the user's next request is admin
admin	`sbx templates quota github:<id> --templates 10 …`	old → new; the user's `sbx quota` shows it
admin	`sbx admin subjects revoke github:<id>`	admin gone at the next request
admin	`sbx admin subjects unenroll github:<id>`	the user's next request: `403`

No settings edit, no setup.sh, no redeploy
every step is an API call that takes effect at the next request and leaves a record

Demo: a template's lifecycle, image to retire

As the demo runs it, as one enrolled user, profile `base` (demo runbook §1-§4)

Step	Command	What the room sees
1 what I may do	`sbx quota`, `sbx policy`, `sbx hosts`	limits and use, profiles, host slots
2 check, then create	`sbx templates create --spec F --dry-run`, then without	local checks pass; a build queued
3 follow the build	`sbx templates follow ID`	queued → building → probing → registered
4 versions	`templates build`, `tag`, `edit`	a second build; a tag names one
5 use	`templates activate --wait`; `sbx create --template ID`; `sbx run`	a lease; a sandbox on that build
6 release, retire	`templates deactivate`; `templates retire --wait`	the worker stops; a later create: `410`

Two refusals, shown between steps 2 and 3; nothing is built

1. Refused by mocactl, before sending
`create --spec F --image <another digest>`
rule `M7.spec_image`: `--image` must match the image named in the spec; refused locally, nothing sent

2. Refused by the control plane's policy
`create --spec F --profile benchmark`
`422 policy_refused`: no trusted gate record; a `check` record

Every step leaves a record on the template's chain: create, gate, build, check, activate, retire.

Five routes, one mocactl command; each change is a record

Reads: any admin. Changes: an admin named in the settings (enrollment spec §4)

Change	Route	`mocactl sbx admin subjects …`	Record
list subjects, roles, source	`GET /v1/subjects`	(no argument)	none
enroll	`PUT /v1/subjects/{s}`	`enroll SUBJECT`	`member`
unenroll	`DELETE /v1/subjects/{s}`	`unenroll SUBJECT`	`member`
grant admin, optional expiry	`PUT …/roles/admin`	`grant SUBJECT --until T`	`role`
revoke admin	`DELETE …/roles/admin`	`revoke SUBJECT`	`role`

Refusals: `409 bootstrap_subject` for a subject the settings name; `409 admin_granted` for an unenroll while an admin grant keeps the subject enrolled; `403` for a non-admin.

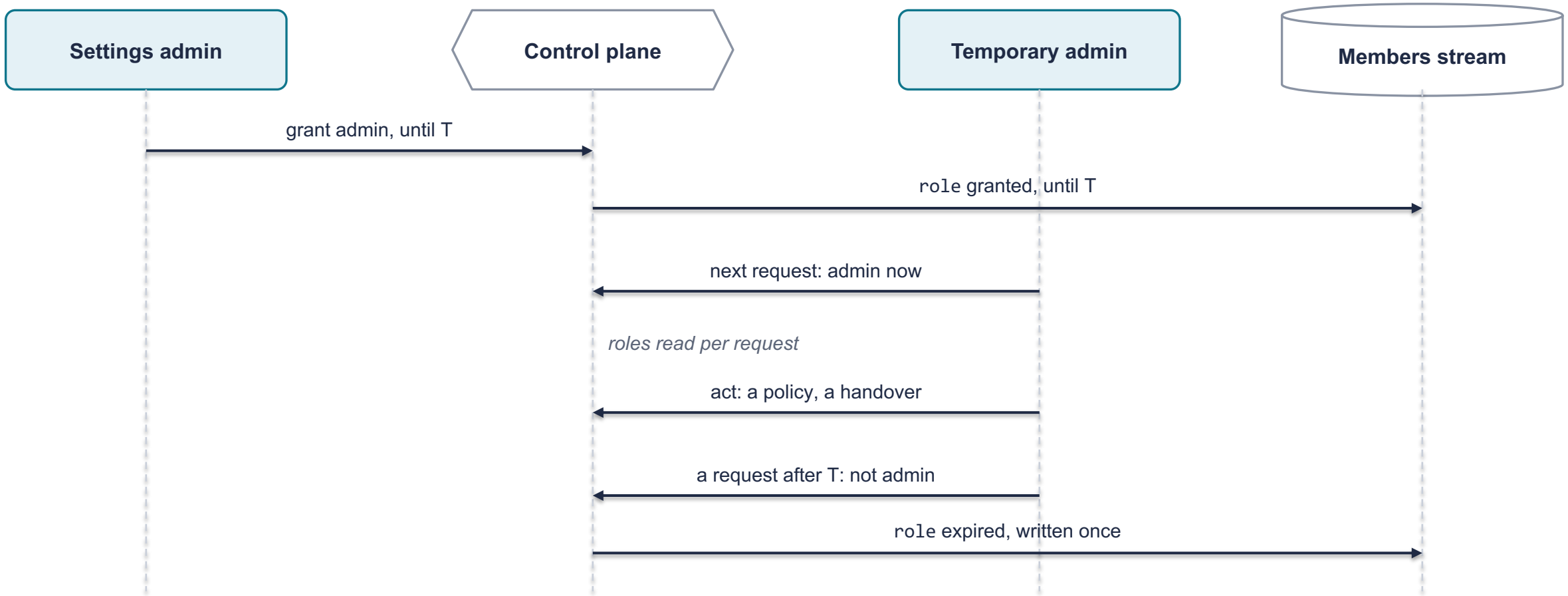
The client checks its target: a login for another control plane is refused before any write; each write prints old → new.

Subjects are login ids

`github:<numeric id>` (or `oidc:<sub>`), from the existing login; never a name

A grant takes effect at the next request, and can expire by itself

A temporary admin for one task, with no setup.sh (enrollment spec §5)



No revoke step and no redeploy: the grant lapses on its own

Onboarding a user: one GitHub login, one enroll

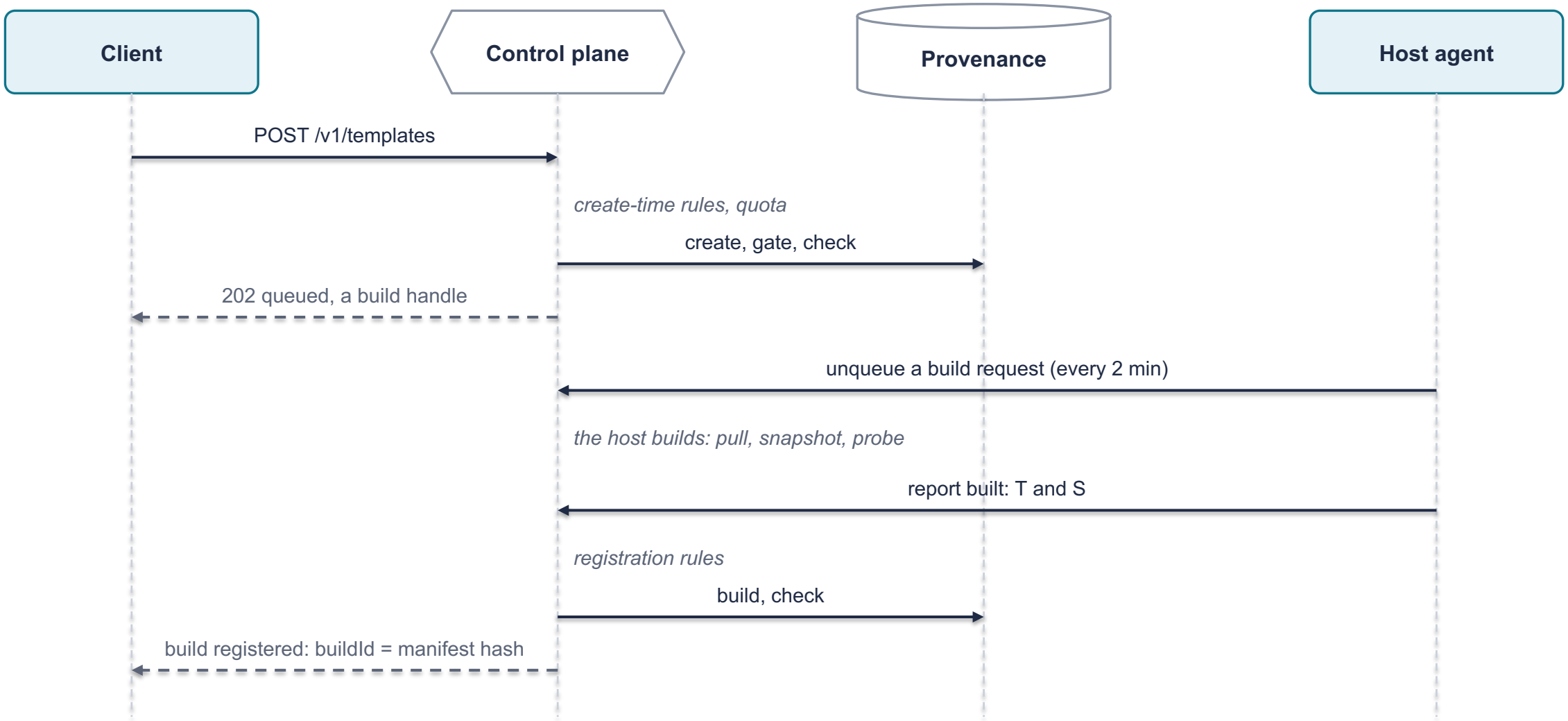
Moca's own device-flow login; no token is pasted, no settings edit

1	**Get a GitHub account**	the subject is its numeric id, `github:<id>`
2	**`mocactl sbx login`**	GitHub's device flow, through the deployment's OAuth app; `403 not_enrolled` until enrolled
3	**An admin enrolls it**	`sbx admin subjects enroll github:<id>`; a `member` record names who did it
4	**The next request works**	`sbx templates`, `sbx quota`, `sbx create`; no redeploy

One login per control plane
each namespace is its own deployment: a user logs in to each one it uses, and is enrolled in each

Create to registered: no approver, every decision recorded

Sequence 1: onboarding (spec §4, §6.1, §8)



Refusals are answers with a rule, not silence

What a client sees when a create cannot proceed (spec §4, §8; G21, G22, G29)

Case	Answer	Recorded
Policy refuses the create	422 `policy_refused` with the rule id	a `check` record
Another subject owns the id	409 `template_exists`	nothing queued
Same inputs as a build a host still holds	200 or 202 with `reused: true`	no second build
Over a quota	429 `quota_exceeded`, naming which	nothing queued
The owner withdraws a build	cancel: `failed` at step `cancelled`	a `build` record
The build or probe fails on the host	`failed`, with step and code (`probe_failed`)	a `build` record

Nothing that names where an image lives leaves the host
failure text is redacted; a record naming the registry is not sent at all (G25, G28)

A create names a profile; the profile names the rules

Policy profiles an admin sets (template spec §8; subject-trust spec §3)

Profile	Gate record	Egress	For
`base`	none	per spec	general images, no benchmark claim
`benchmark`	trusted by an admin (X1)	none (X11)	curated benchmark tasks
`benchmark-self`	the creator's own (X1, self)	none (X11)	a user's own benchmark tasks
`general-self`	the creator's own (X1, self)	the host's allowlist	general users who need a network

Checked at create and at registration: the id; the image, by digest, from an allowed registry and repository; the spec; the size envelope; each rule the profile names.

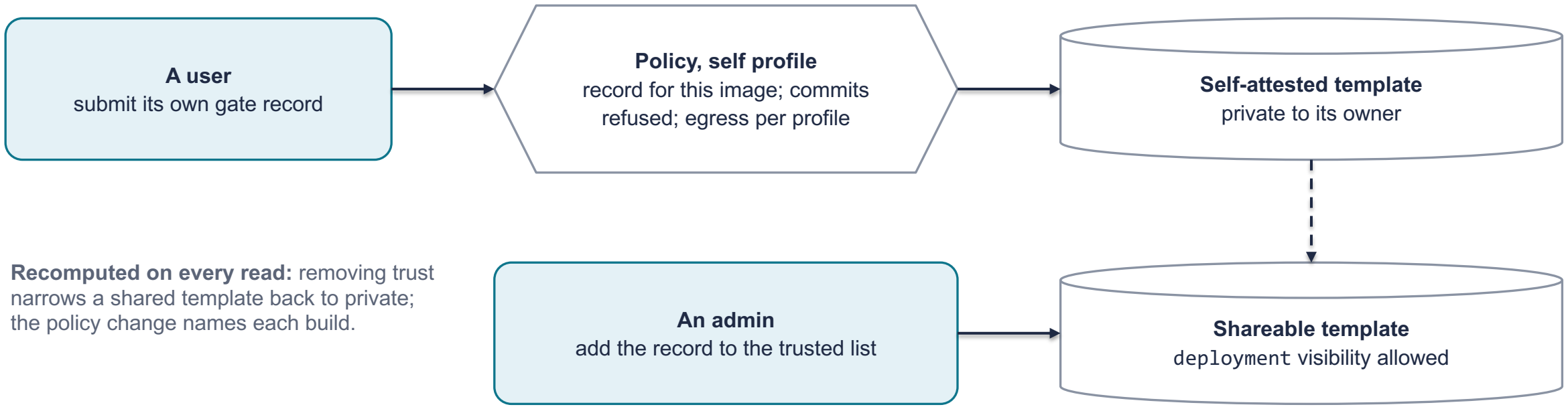
Quotas per subject: templates owned, builds queued, templates active at once; `mocactl sbx quota` shows limit and use.

A refusal names its rule

`422 policy_refused` with the rule id, and a `check` record on the caller's own template's stream

Your own gate record counts for your own templates, privately

Trust comes from who submits a record, never from what it says (subject-trust spec §2-§6)



Asking for `deployment` visibility before trust
is refused `409 self_attested`, on the create, on a new build, and on an edit

The client's whole path, in mocactl

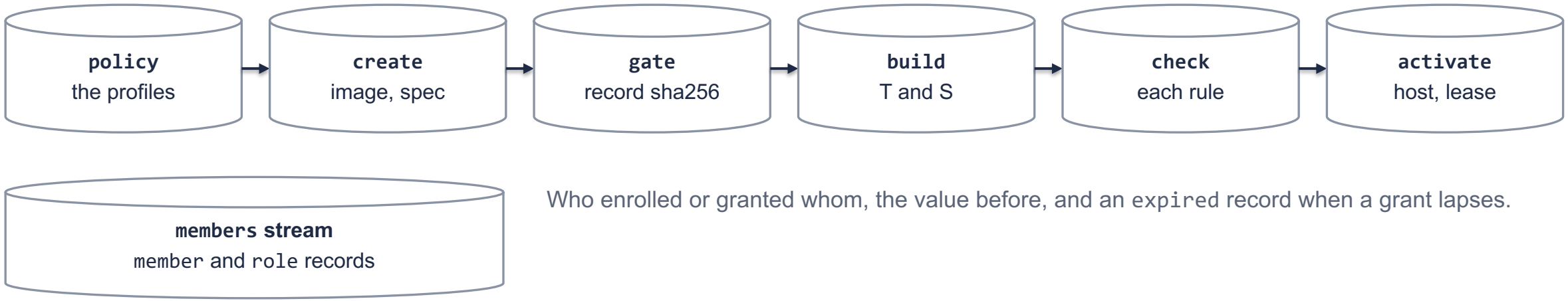
Each step a command, `--json` on all of them (mocactl template commands spec §3, §4a)

Step	Command	What it does
before a create	`sbx quota`, `sbx policy`	limits and use; the profiles and rules
check locally	`sbx templates create --dry-run`	the rules the inputs decide; creates nothing
create and follow	`sbx templates create`, `follow`	post the spec and gate record; wait for registered
new build, cancel	`sbx templates build`, `cancel`	a build of an existing template
activate, use	`sbx templates activate --wait`, `sbx create --seed`	lease a slot; seed the workspace
end of life	`sbx templates retire --wait`	until no host holds the build
users and admin	`sbx admin subjects …`	print old → new; detect a concurrent write
a record per call	`--record F`	one line per HTTP attempt, no secret in it

Errors keep the server's reason
the rule id, the quota hit, the readiness and the queue position; exit 5 for a wait that timed out

Every step is a chained record, checkable offline

Provenance (template spec §7; enrollment spec §8): digests, not signatures



- **One file per record**, named by the sha256 of its canonical JSON; each names its predecessor. Any edit, drop or reorder breaks the chain.
- **Verified offline** by a stdlib Python script or the same rules in TypeScript, from an export, on a machine with no access to the cluster.

An export proves itself consistent, not complete
compare each stream's head with a copy kept elsewhere to catch a dropped tail

A build has two states: how far from usable, and whether it runs now

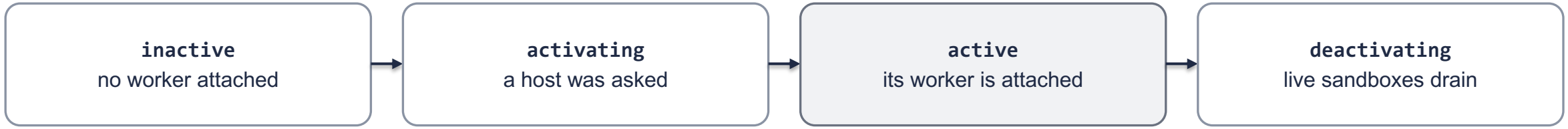
Onboarding is set by the host's reports and policy; readiness by what is attached

Onboarding (spec §3.3)



Off the path: `failed` (a build or probe failure, with its step; or the owner's cancel), `rejected` (policy at registration, with its rule), `retired`.

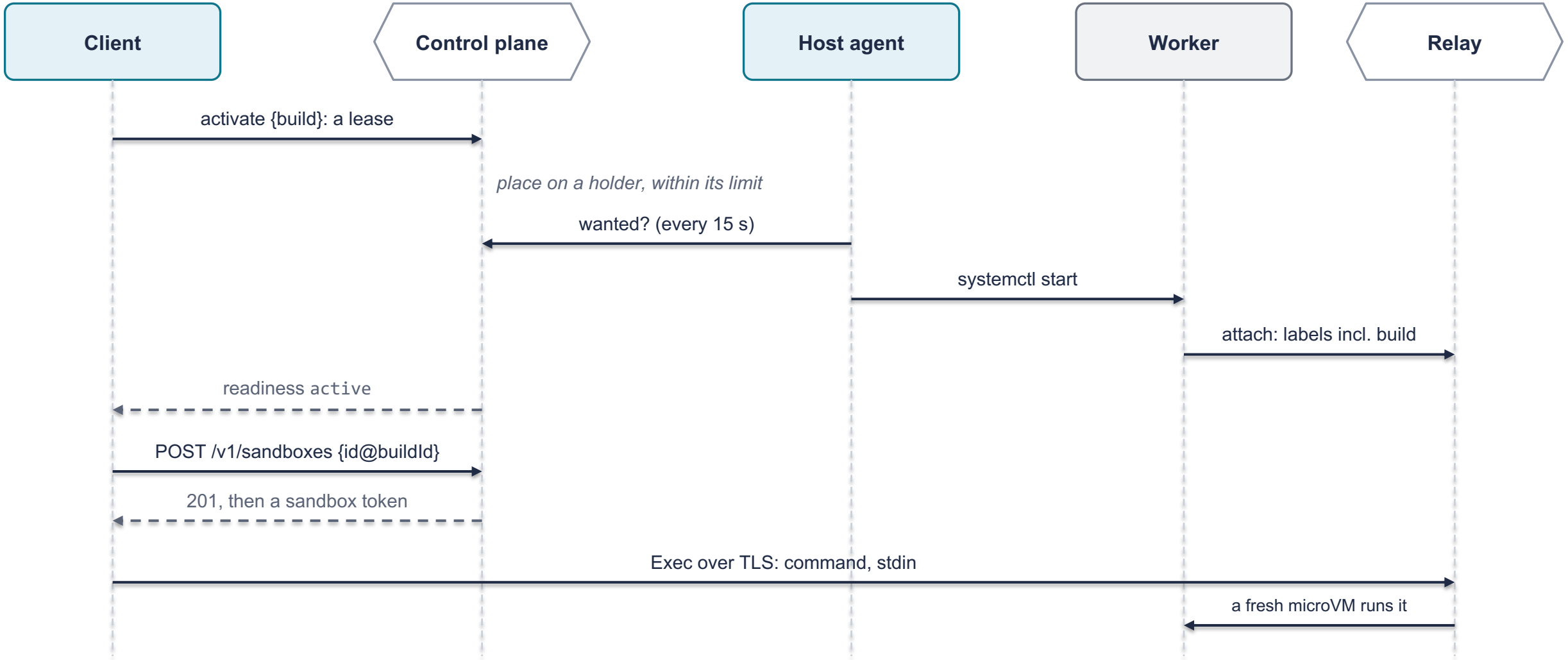
Readiness of a registered build



`unavailable`: no host holds its snapshot, or every holder is full. `active` is derived from the relay's attached record, so the state cannot claim a worker that is not there.

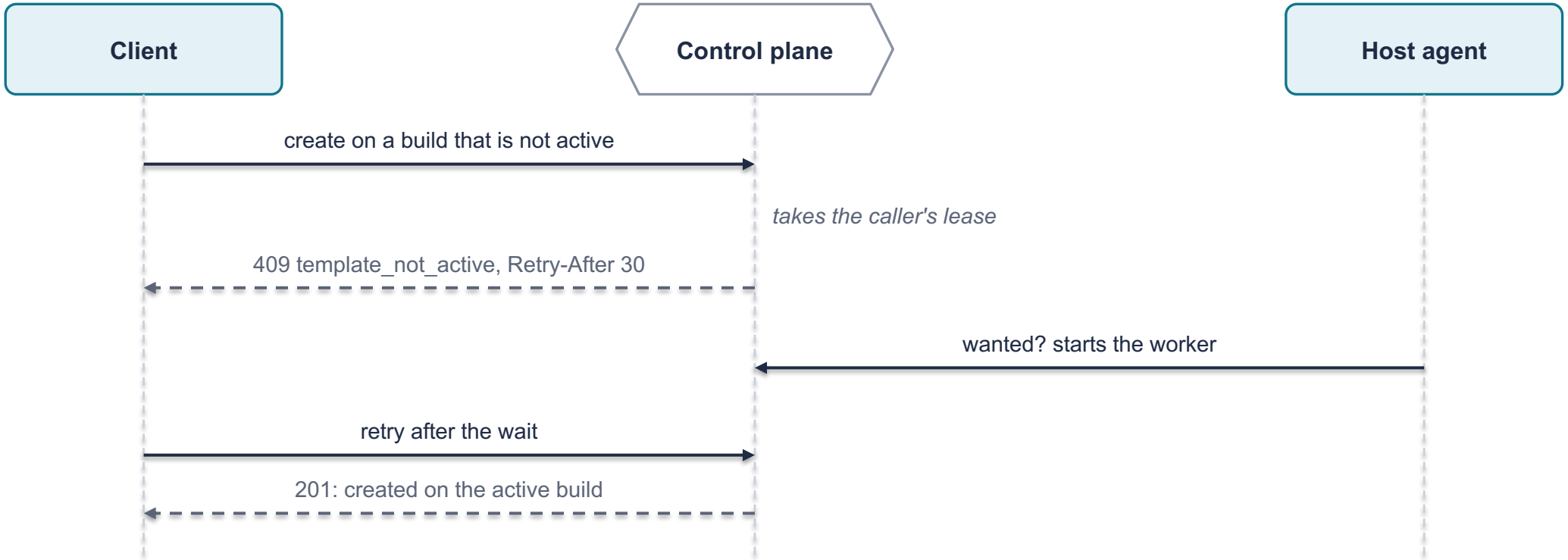
Activation is a lease; the worker starts in seconds

Sequence 2: activate, then a sandbox on the pinned build (spec §6.3; G31)



First use: name a template and go

Activation on first use (spec §6.4), for sandboxes and sessions alike

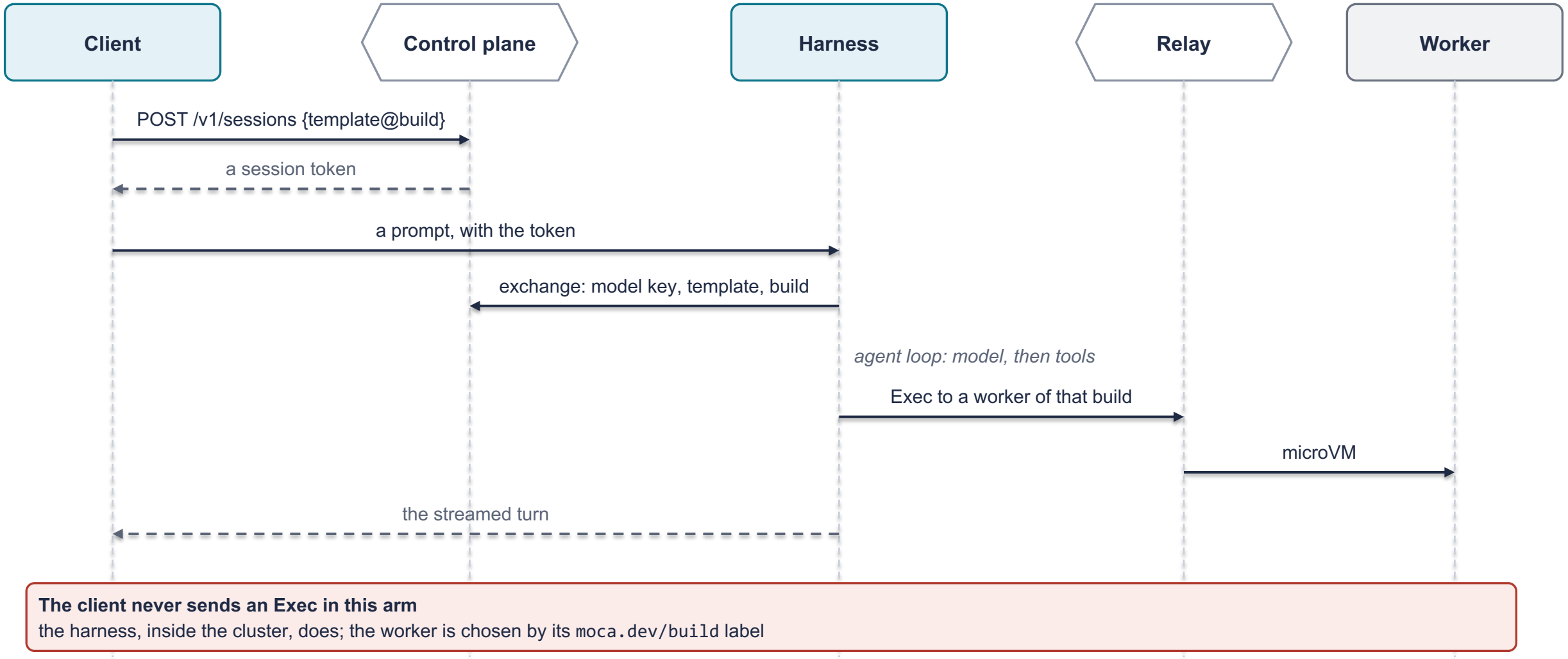


`activate: false` starts nothing. No sandbox is kept while waiting, so a wait never uses the caller's sandbox cap. `mocactl` retries for up to 10 minutes, saying why.

Without host agents nothing changes: the create succeeds as before

A Moca Pi session runs only on its build

Sequence 3: a session pinned to a build (combined control plane §3.4; G19)



One control plane serves both kinds of brain

The combined route profile (combined control plane design, C1-C4)



Decision	Chosen
C1 How	a third profile, `combined`; `harness` and `sandbox` unchanged
C2 Login	open as on `harness`; enrollment checked on sandbox routes and template writes
C3 Where	a new namespace, so the validated SBX0 stays as it is
C4 Turns by build	in: a worker label and a filter (G19)

The API covers the whole lifecycle, create to retire

Each activity a route and a mocactl subcommand; each writes a record naming who

Activity	Who	Record it writes
create a template, or a new build	any enrolled client	create, gate, check
build, probe, register by policy	the host agent, then policy	build, check
activate, deactivate (a lease)	the owner; anyone, if shared	activate, deactivate
move a tag onto a new build	the owner	tag (new and old build)
edit description, visibility	the owner	visibility (new and old)
retire a template or a build	the owner	retire (and the host's removal)
reject a build after the fact	an admin, with a reason	reject
transfer ownership	an admin	owner
set quotas, default or per user	an admin	policy

Deployed and shown end to end
a retire never cuts off running work: refused while a sandbox or session uses the build

Four windows validated the lifecycle end to end

Spec §10 steps, as run 2026-11-04 to 2026-11-05 (no model)

Step	Window 1	Window 2	Windows 3-4
Create, build, register	all four	all four, fresh	all four, each
Refused create (X10)	yes	yes	yes
Broken probe	not reached	`probe_failed`	step `probe` (w4)
First use	not built	409, then active	as w2
Served build = pinned	every sandbox	every sandbox	every sandbox
Patches match recordings	not compared	every patch	every patch
Offline verification	83 records	140 records, all chains verified	not repeated

Step 3 (a second client on the same instance) passed separately with a second enrolled subject: its create and its new build `409`, its cancel `403`.

Create to retire, through the API alone, on the cluster

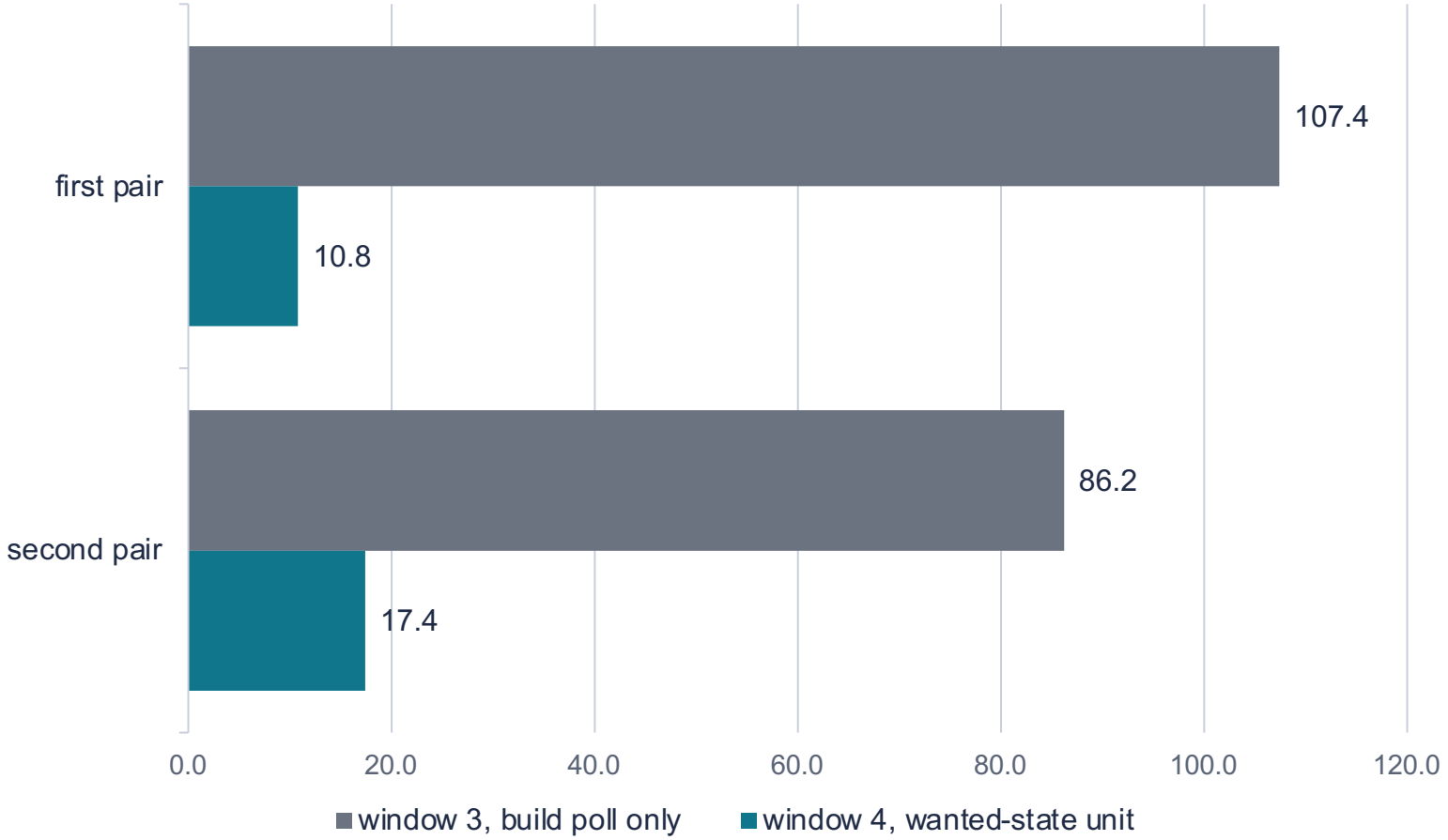
The lifecycle window, 2026-11-06: a user with no role; no operator step (no model)

Stage	What happened
create, build, register	all four by policy; the bad gate refused (X10); the bad probe failed
activate, first use, use	a lease in 23-26 s; every sandbox on its pinned build
retire	every build; a later create refused `410`; readiness `retired` in ~1 s
the host removes it	the KVM host's agent, by itself; `heldBy` emptied in 17-32 s
provenance	every chain before is an unchanged prefix after; 84 records appended
composition	an independent composer: every link of each rep unique

A whole-template retire is every build
earlier windows' builds of those templates were retired too: a new build is the way back

Activation fell from ~100 s to ~11-17 s

Seconds from each activation's request to `active`, both builds of a pair together (G31)



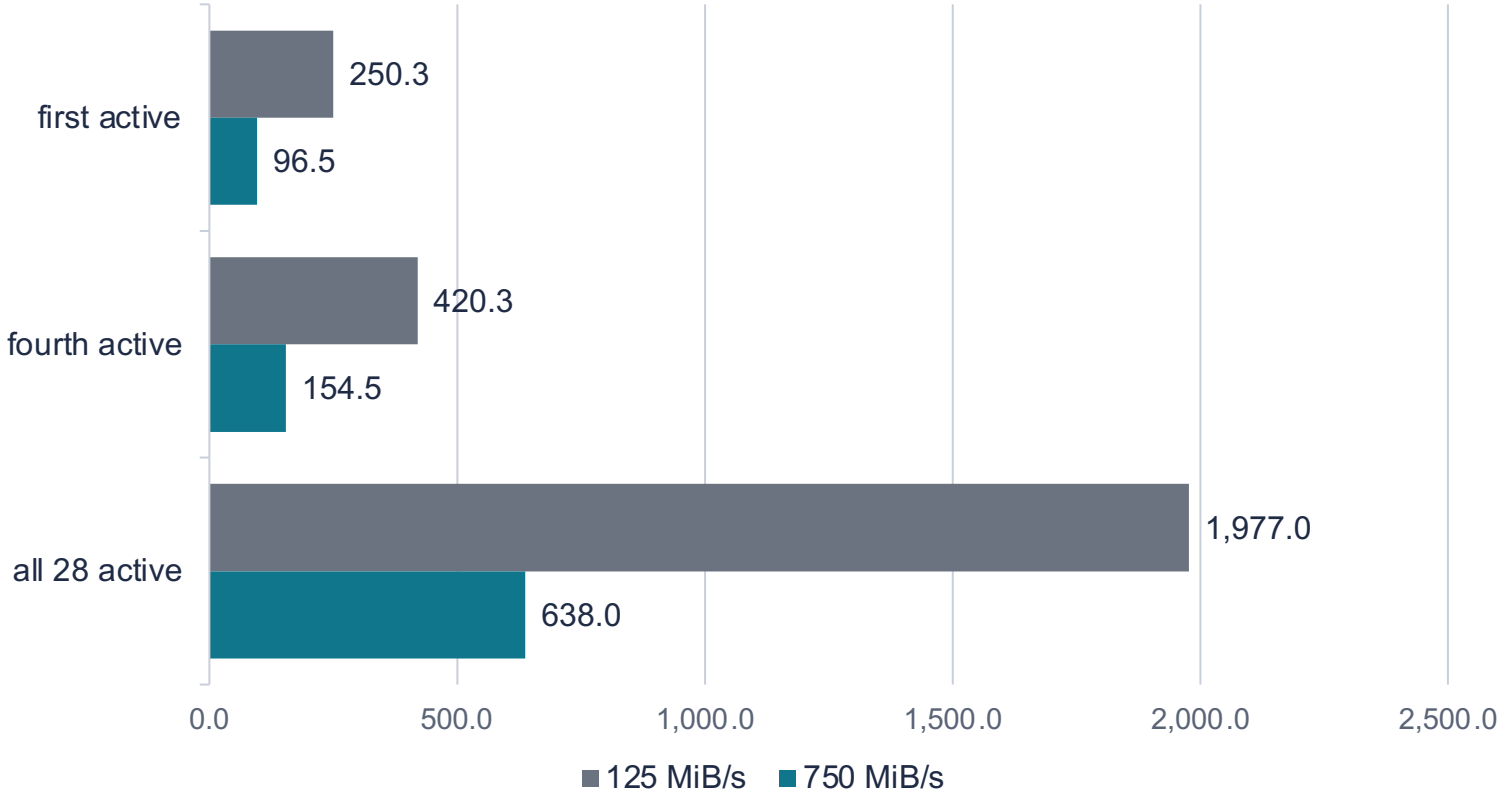
Before: an activation waited for the build poll, which starts 2 minutes after the previous one ends, and behind any build.

After: a separate unit applies the wanted state every 15 s; a lock keeps it and the build poll from acting at once.

One pair per window
a measurement, not a distribution

Disk throughput sets the cold start: 3.1× faster

Seconds until active, 28 activations posted at once, same builds; only the volume's throughput changed (I28b-7-d, D1 and D1b, 2026-11-11)



Mean disk read: 116.1 → 364.7 MiB/s over the round.

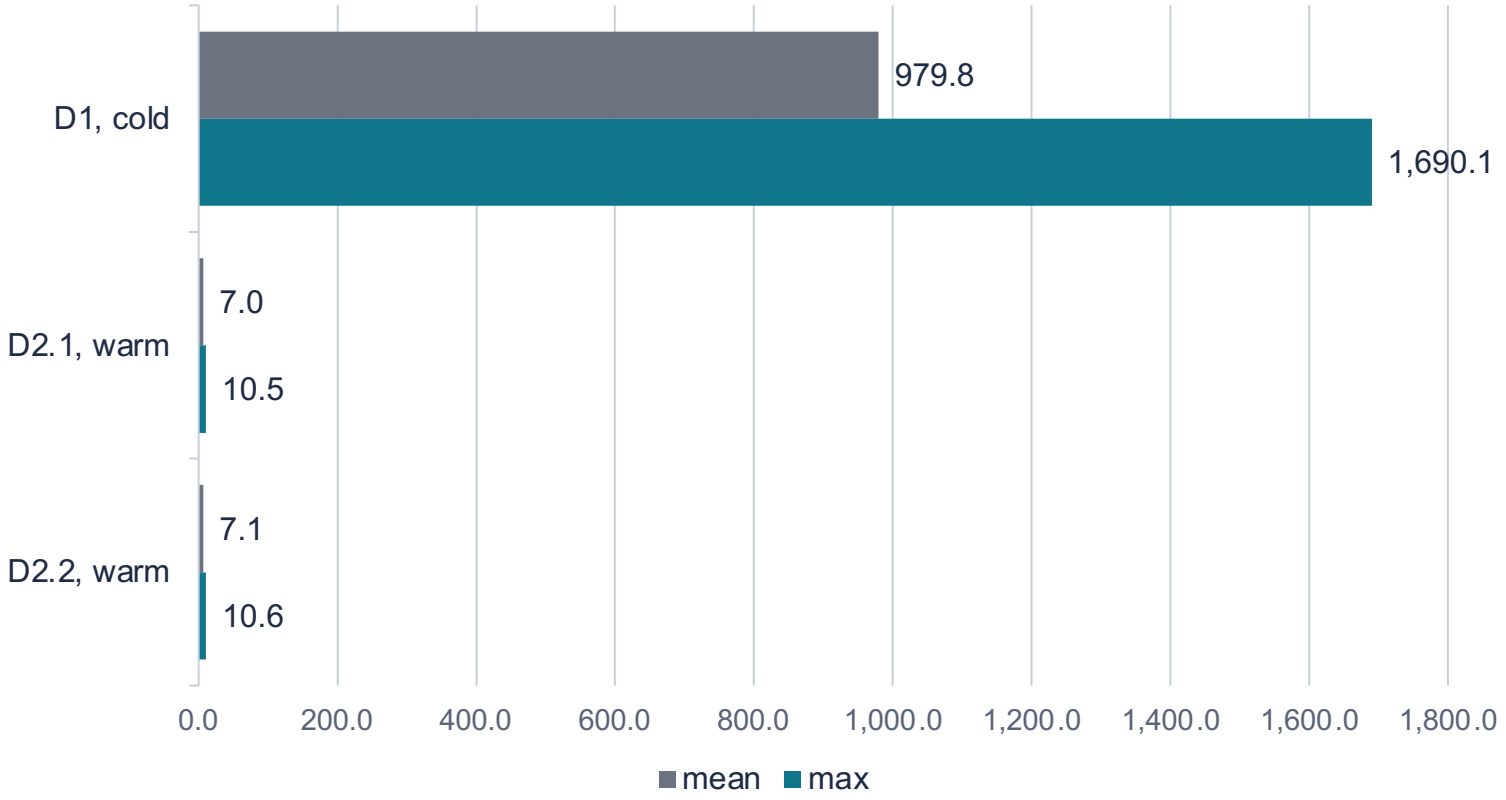
Time per read: 6.99 → 1.65 ms, at 3,000 IOPS both times.

Each cold start read about its whole snapshot
a mean of 8.70 GiB per start at 750 MiB/s: reading less per start is the next lever

one KVM host, 28 SWE-bench Lite templates (4 GiB, 2 vCPUs, snapshots 7.3-13.6 GiB), start limit 4, page cache dropped before each cold round

A warm worker activates in seconds: 7 s, not 980 s

Mean seconds per activation, 28 posted at once, three rounds back to back (l28b-6-d, 2026-11-10/11, at 125 MiB/s)



Warm by the idle grace (900 s): a released worker stays attached, so the next activation needs no restore from disk.

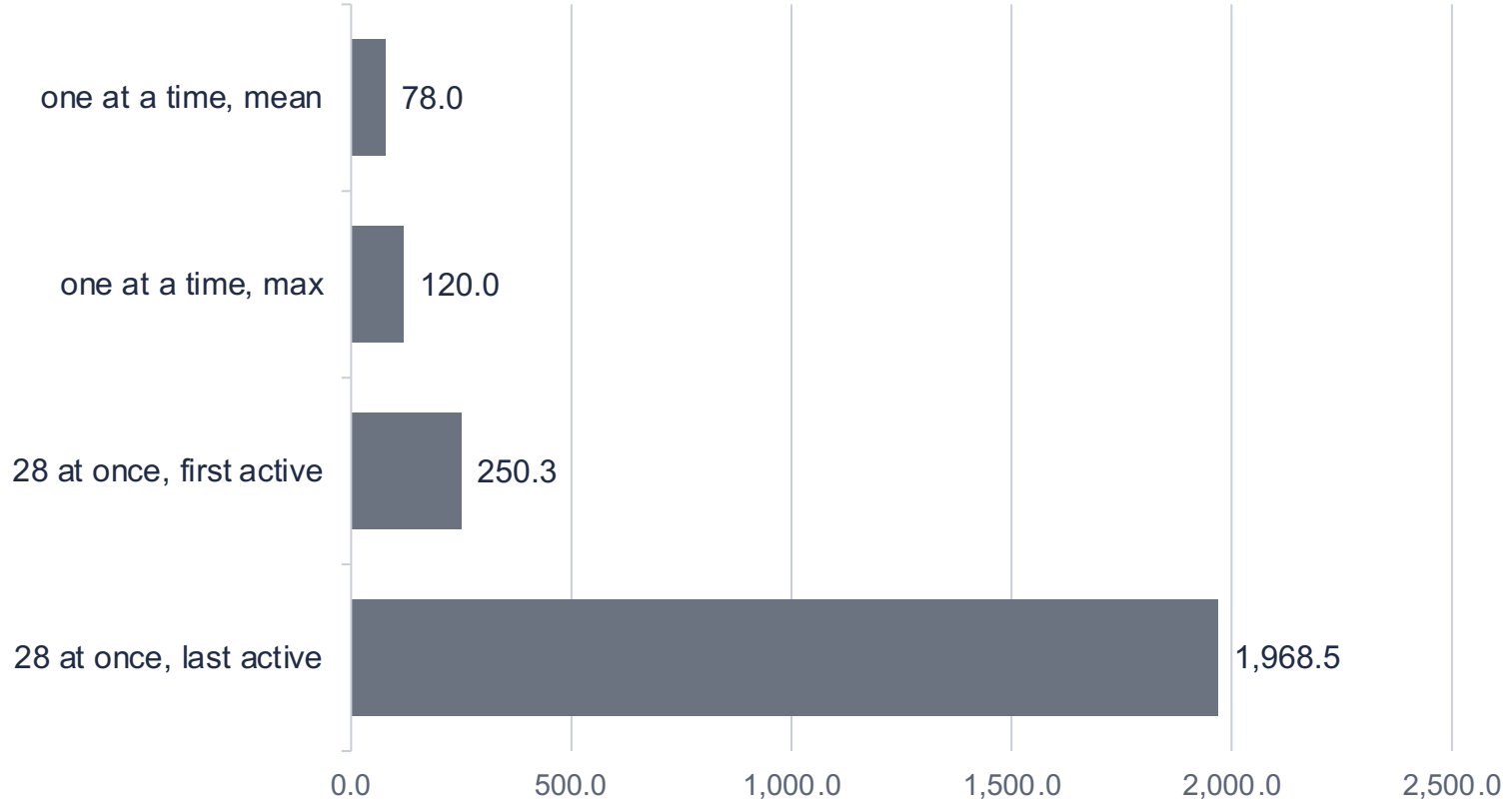
All 28 done after: 1,809 s cold; 19 s in each warm round.

D1's mean covers 23 activations
the client stopped waiting on 5, which the control plane completed; in both warm rounds all 28 workers were still attached

one KVM host, 28 SWE-bench Lite templates (4 GiB, 2 vCPUs, snapshots 7.3-13.6 GiB), start limit 4, page cache dropped before each cold round

28 cold starts at once queue behind the start limit

Seconds until active; alone, a cold start takes about 78 s (125 MiB/s, start limit 4; l28b-3-d serial-1, 2026-11-09/10; l28b-7-d D1, 2026-11-11)



Four start at a time: the rest wait, so the last of 28 is active after about 33 minutes.

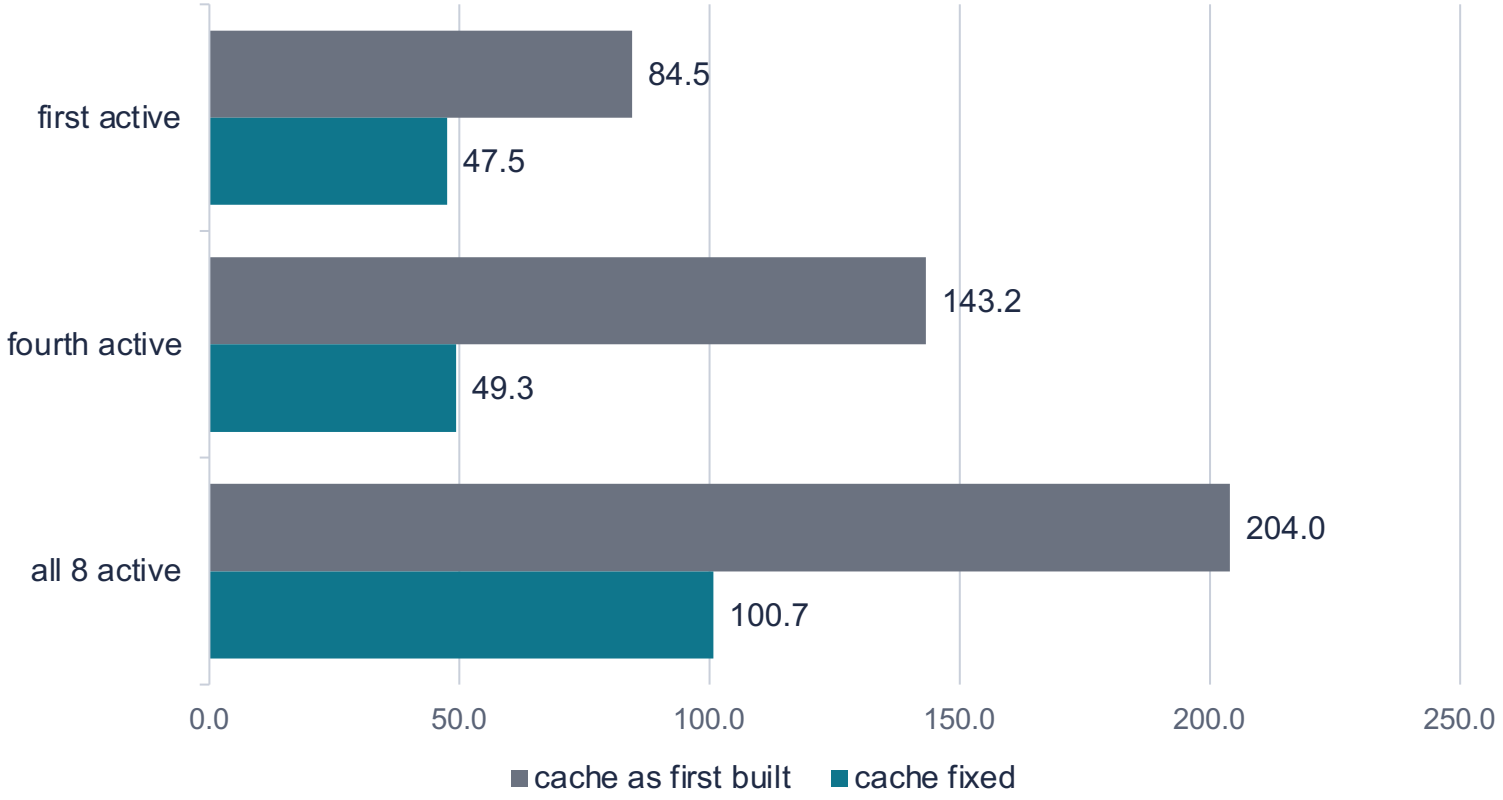
Then the disk: four starts at once share one volume's throughput (previous charts).

One pass per condition
a measurement, not a distribution

one KVM host, 28 SWE-bench Lite templates (4 GiB, 2 vCPUs, snapshots 7.3-13.6 GiB), start limit 4, page cache dropped before each cold round

The fixed verify cache halves a cold start's reads

Seconds until active, 8 cold activations posted at once (G50 window, 2026-11-12, 750 MiB/s, start limit 4)



Read per start: 8.99 → 4.00 GiB, the memory file alone, which a start still pins.

Files re-hashed per start: 4 of 4 → 0 of 4.

Why the first build missed

each VM restore links and chowns the files into its jail, which moved the ctime the cache keyed on

one KVM host, 8 SWE-bench Lite templates (4 GiB, snapshots 7.3-13.5 GiB), page cache dropped before each round

SWE-bench Lite on templates, in groups, retire between

Lite-28 on templates, 2026-11-06/07: stock brains, the sandbox profile, a real model

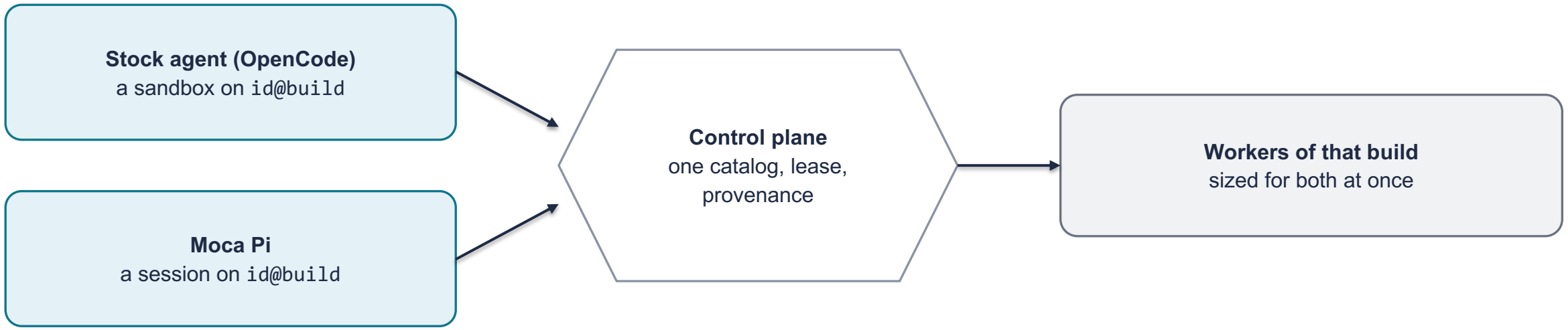
- **Self-serve, in three groups:** each group created, built and registered by policy, used on pinned builds, then retired; the host agent's removal checked before the next group.
- **Provenance:** every chain before an unchanged prefix after; an independent composer joined every link of every rep uniquely.
- **One instance left out:** its id is a template the deployment declares, which blocks a self-serve create under that id (G34).

Two agents on one template at once
the one-session minimum budget refused some commands at admission (G36): clients now retry and exit 75; budgets sized for the work run at once

Removal keeps workspaces
for the janitor; without it they stay on the host's disk

Two brains on one build, every link exact

The Moca Pi window, 2026-11-07: a stock agent and Moca Pi, a real model, the combined control plane



What the session now does	Seen
seeded from its build's own spec	`sandbox.seed.source: build`, the build = the pin
returns the workspace's patch with the turn	whole, its sha256 and size matching
leaves a `session` record at create and delete	its lease and build, on the template's chain
composes like a sandbox	every link of every rep unique, for both brains

What the windows found, and the fixes

Every finding has a register entry; Moca's are fixed and validated

Id	Found	Fixed in	Validated
G26	commands over 32 KiB hung the worker	the worker: refused at once	w2-w4
G25, G28	a registry host left the host in a reason	host agent redaction	w2-w4
G21, G29	duplicate builds; reuse of a removed snapshot	control plane reuse rules	w2-w4
G31	activation waited for the build poll	the wanted-state unit	w4
G19	a session could run on another build	the build label and filter	combined
G33	a rep's lease found only by time	sandbox records by digest	lifecycle
G35	a session got no seed, no patch, no record	seed from the build; patch; `session`	Moca Pi
G36	commands refused at a one-session budget	client retry; budget sizing	Moca Pi
G30	~20 s client stalls	the client's DNS (not Moca)	w4: 0 stalls

Since fixed
G34: an admin hands a declared template to a subject; G37: mocactl's template option takes `id@build`

The enhancements are open upstream, as reviewable PRs

rossoctl/moca, 2026-11-12: each stacked where it builds on another

Area	PRs	What it brings
Users	#762	enroll subjects and grant the admin role through the API
Templates	#760	a template store with builds, tags and readiness
	#766	self-serve create, gated by policy, with quotas
	#769	a subject's own gate record, for its own templates
	#759	a Moca Pi session can choose a sandbox template
Provenance	#756	template provenance in a hash chain, verifiable offline
mocactl	#767, #768	authoring commands; `--record`; token renewal
	#761, #755	retry a refused command (exit 75); `--version`
Hosts	#757, #758	microVM worker and snapshot fixes; deploy fixes
	#771, #772	from an image to a registered template; the host agent
	#773	a verify cache: a start re-hashes only changed snapshot files

Each one reviewed before its push
a tree check, a deploy-input diff, and a scan for hosts, ids, tokens and results

Next: land the PRs upstream, then the demo

In order, each with the user's decision first

1	**Upstream review**	answer the reviews of the open PRs; land them in stack order
2	**The next PRs**	activation, removal, memory, pacing and egress (#742-#745, #752), on the host agent (#772)
3	**The demo, rescheduled**	self-serve template and user lifecycle, live, on the synced code

Done since
synced with upstream main and tested on both namespaces; roles through the API on both; the client's whole path in mocactl

Users and templates both needed an operator; now both are API calls

Where the work started (template spec §1-§2; enrollment spec §1)

- **Templates:** each was a hand-run build, a hand-picked worker id, a settings edit and a redeploy; benchmarks need 30+ of them, built and retired by clients, not operators.
- **Users:** who may log in, and who is admin, were deployment settings; a change was a settings edit and a `setup.sh` run, and left no record.
- **One admin action took two cluster writes:** add a temporary admin by `setup.sh`, act through the API, remove the admin by `setup.sh` again.
- **And proof:** which image, which checks, which snapshot, and who changed whose access.

